МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп’ютерних технологій

Звіт

З ЛАБОРАТОРНОЇ №1

Варіант 12

на тему: **"Основи функціонального програмування у Python"**

Виконав:

Студент групи ФЕП-22

Сисак Ілля

Перевірив:

Аспірант

Остріков Данило Андрійович

Львів 2026

Мета. Ознайомитися з парадигмою функціонального програмування в мові Python. Навчитися виявляти та ізолювати побічні ефекти (I/O, глобальний стан, мутації), будувати чисте обчислювальне ядро з дотриманням принципів детермінованості та референтної прозорості, а також застосовувати функції вищого порядку та typing.Callable для параметризації бізнес-політик на прикладі системи складського обліку (Варіант 12).

2. Теоретичні відомості

- **Чиста функція (Pure Function):** Функція, яка для однакових вхідних аргументів завжди повертає однаковий результат (детермінованість) і не викликає побічних ефектів (не змінює глобальний стан, не мутує вхідні аргументи, не виконує операцій введення/виведення).
- **Референтна прозорість (Referential Transparency):** Властивість виразу або виклику функції, за якої його можна замінити обчисленим значенням у будь-якому місці програми без зміни поведінки системи.
- **Побічні ефекти (Side Effects):** Взаємодія із зовнішнім середовищем або зміна стану поза межами локального контексту функції (виклики print(), читання/запис файлів, звернення до поточного часу time.time(), генерація випадкових чисел, мутування вхідних структур даних).
- **Ізоляція ефектів (Functional Core / Imperative Shell):** Архітектурний підхід, при якому вся обчислювальна та логічна обробка даних зосереджена в чистому ядрі (core.py), а всі побічні ефекти (введення, консольне виведення, робота з таймерами) винесені в зовнішню оболонку (app.py).
- **Параметризація та композиція через typing.Callable:** Передача алгоритмів та бізнес-правил (політик знижок, податків, критеріїв дозамовлення) у вигляді функцій-аргументів, що дає змогу будувати гнучкі та компоновані обчислювальні конвеєри за допомогою функцій вищого порядку (наприклад, compose(f, g) та make_multiplier(k)).

Хід роботи

Проєктування типів даних (core.py):

Створено структуру товару Product через TypedDict з полями id, name, category, stock, min_stock, unit_price, reorder_qty, total_cost, timestamp.

Створено структуру InventoryResult для строгої статичної типізації поверненого результату обробки.

```python
from copy import deepcopy
from typing import Callable, Dict, Iterable, List, Optional, Tuple, TypeVar, TypedDict

class Product(TypedDict, total=False):
    id: int
    name: str
    category: str
    stock: int
    min_stock: int
    unit_price: float
    reorder_qty: int
    total_cost: float
    timestamp: float

class InventoryResult(TypedDict):
    count: int
    total_cost: float
    products: List[Product]

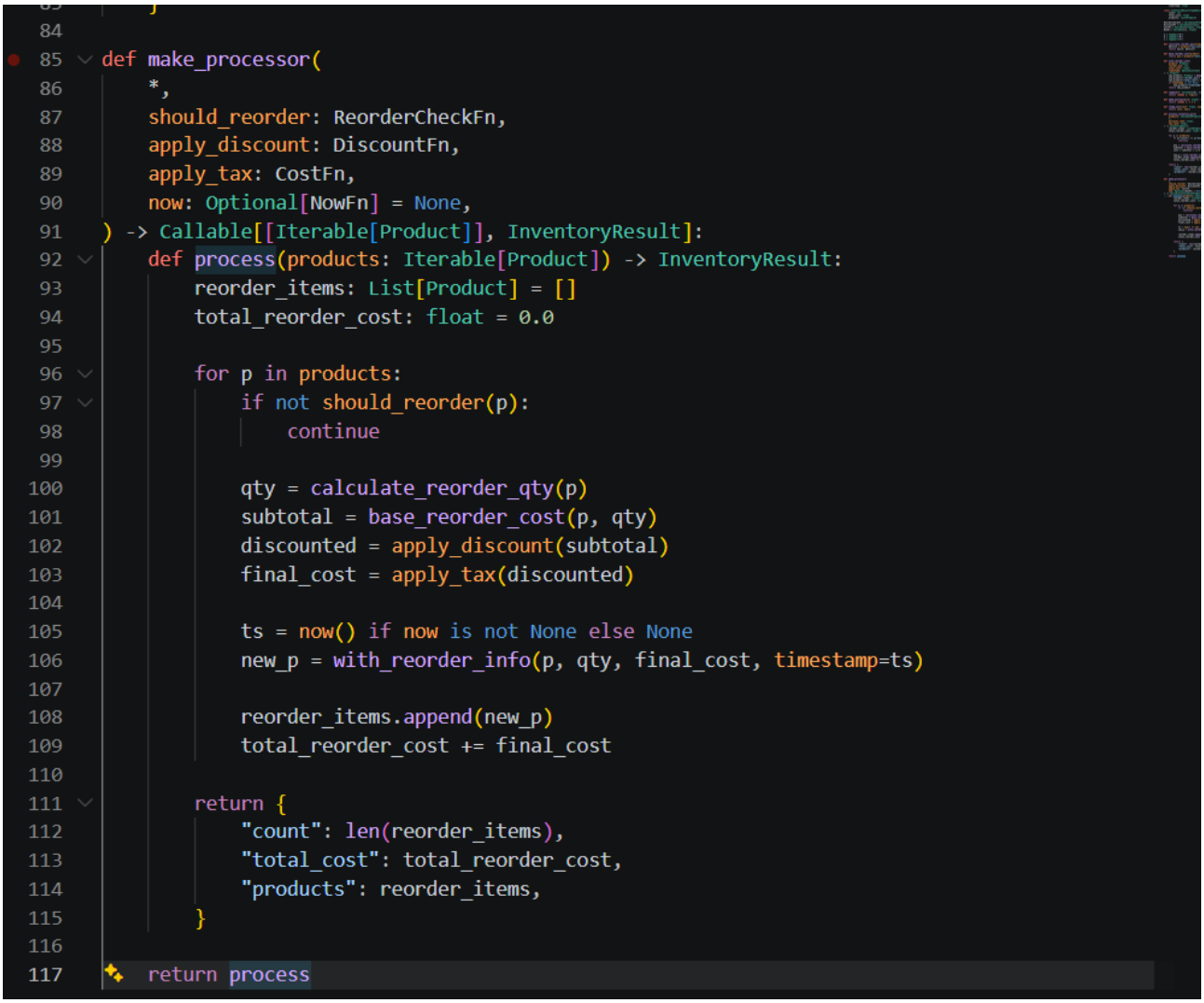
ReorderCheckFn = Callable[[Product], bool]
DiscountFn = Callable[[float], float]
CostFn = Callable[[float], float]
NowFn = Callable[[], float]

A = TypeVar("A")
B = TypeVar("B")
C = TypeVar("C")

def calculate_reorder_qty(product: Product) -> int:
    deficit = product["min_stock"] - product["stock"]
    return max(0, deficit)

def base_reorder_cost(product: Product, qty: int) -> float:
    return qty * product["unit_price"]

def with_reorder_info(
    product: Product
```



Розробка чистого обчислювального ядра (core.py):

1. Реалізовано чисті функції calculate_reorder_qty (обчислення дефіциту товару) та base_reorder_cost (розрахунок базової вартості).
2. Створено чисту функцію with_reorder_info, яка використовує deepcopy для повернення нового об'єкта товару без мутації оригінального словника.
3. Реалізовано базову чисту функцію process_inventory_pure з явною передачею параметрів discount_rate та tax_rate.
4. Реалізовано функцію вищого порядку make_processor, яка приймає політики у вигляді Callable (should_reorder, apply_discount, apply_tax, now) та повертає налаштовану функцію-обробник.
5. Додано функцію композиції compose(f, g) та фабрику множників make_multiplier(k) для комбінування декількох знижок.
6. Реалізовано функцію stamp_cost(cost, now) для інжекції зовнішньої залежності часу через Callable.



```python
def with_reorder_info(
    product: Product,
    reorder_qty: int,
    total_cost: float,
    timestamp: Optional[float] = None,
) -> Product:
    new_product: Product = deepcopy(product)
    new_product["reorder_qty"] = reorder_qty
    new_product["total_cost"] = total_cost
    if timestamp is not None:
        new_product["timestamp"] = timestamp
    return new_product

def compose(f: Callable[[B], C], g: Callable[[A], B]) -> Callable[[A], C]:
    return lambda x: f(g(x))

def make_multiplier(k: float) -> Callable[[float], float]:
    return lambda x: x * k

def stamp_cost(cost: float, now: NowFn) -> Tuple[float, float]:
    return cost, now()

def process_inventory_pure(
    products: Iterable[Product],
    *,
    discount_rate: float,
    tax_rate: float,
) -> InventoryResult:
    reorder_items: List[Product] = []
    total_reorder_cost: float = 0.0

    for p in products:
        if p["stock"] >= p["min_stock"]:
            continue

        qty = calculate_reorder_qty(p)
```

Розробка оболонки побічних ефектів (app.py):

1. Винесено консольне виведення підсумкового звіту в окрему функцію render_report.
2. У функції run сформовано тестовий набір товарів та продемонстровано запуск чистого ядра з використанням параметризованих функціональних політик та системного часу time.time.

```python
import time
from typing import List
from core import (
    Product,
    InventoryResult,
    compose,
    make_multiplier,
    make_processor,
    process_inventory_pure,
)

def render_report(result: InventoryResult) -> None:
    print("=" * 65)
    print("          ЗВІТ СКЛАДСЬКОГО ОБЛІКУ (ДОЗАМОВЛЕННЯ)          ")
    print("=" * 65)
    for p in result["products"]:
        ts_str = f" | time: {p['timestamp']:.2f}" if "timestamp" in p else ""
        print(
            f" ID: {p['id']:<3} | {p['name']:<15} | Qty: {p['reorder_qty']:>3} | Cost: ${p['
        )
    print("-" * 65)
    print(f" Позицій для дозамовлення : {result['count']}")
    print(f" Загальна вартість       : ${result['total_cost']:.2f}")
    print("=" * 65)

def run() -> None:
    sample_products: List[Product] = [
        {"id": 1, "name": "Клавіатура", "category": "Периферія", "stock": 3, "min_stock": 1
        {"id": 2, "name": "Монітор", "category": "Дисплеї", "stock": 15, "min_stock": 5, "u
        {"id": 3, "name": "Мишка", "category": "Периферія", "stock": 2, "min_stock": 15, "u
        {"id": 4, "name": "Кабель HDMI", "category": "Аксесуари", "stock": 8, "min_stock":
    ]

    res1 = process_inventory_pure(sample_products, discount_rate=0.05, tax_rate=0.2)
    render_report(res1)
```

```python
def run() -> None:
    res1 = process_inventory_pure(sample_products, discount_rate=0.05, tax_rate=0.2)
    render_report(res1)

    bulk_discount = make_multiplier(0.85)
    partner_discount = make_multiplier(0.95)
    combined_discount = compose(partner_discount, bulk_discount)

    tax_policy = make_multiplier(1.20)
    reorder_policy = lambda p: p["stock"] < p["min_stock"]

    processor = make_processor(
        should_reorder=reorder_policy,
        apply_discount=combined_discount,
        apply_tax=tax_policy,
        now=time.time,
    )

    res2 = processor(sample_products)
    render_report(res2)

if __name__ == "__main__":
    run()
```

Автоматизоване тестування та static type checking (tests/test_core.py):

1. Написано unit-тест test_referential_transparency для перевірки детермінованості ядра (однаковий вхід → однаковий вихід).
2. Написано unit-тест test_no_mutation для підтвердження відсутності мутацій вхідного списку товарів.
3. Написано unit-тест test_callable_policies для перевірки розрахунку вартості при передачі політик через Callable.
4. Написано unit-тести для перевірки композиції test_compose_and_multiplier, інжекції часу test_stamp_cost_dependency_injection та чистоти оновлення test_with_reorder_info_purity.
5. Налаштовано pytest.ini з конфігурацією pythonpath = . для автоматичного вирішення шляхів імпорту.

```python
from copy import deepcopy
import pytest
from core import (
    Product,
    compose,
    make_multiplier,
    make_processor,
    process_inventory_pure,
    stamp_cost,
    with_reorder_info,
)

@pytest.fixture
def sample_products() -> list[Product]:
    return [
        {"id": 1, "name": "Клавіатура", "category": "Периферія", "stock": 3, "min_stock": 1
        {"id": 2, "name": "Монітор", "category": "Дисплеї", "stock": 10, "min_stock": 5, "u
        {"id": 3, "name": "Мишка", "category": "Периферія", "stock": 0, "min_stock": 5, "un
    ]

def test_referential_transparency(sample_products: list[Product]) -> None:
    args = {"discount_rate": 0.1, "tax_rate": 0.2}
    r1 = process_inventory_pure(sample_products, **args)
    r2 = process_inventory_pure(sample_products, **args)
    assert r1 == r2

def test_no_mutation(sample_products: list[Product]) -> None:
    original = deepcopy(sample_products)
    process_inventory_pure(sample_products, discount_rate=0.0, tax_rate=0.0)
    assert sample_products == original

def test_callable_policies(sample_products: list[Product]) -> None:
    should_reorder = lambda p: p["stock"] < p["min_stock"]
    apply_discount = lambda cost: cost * 0.9
    apply_tax = lambda cost: cost * 1.2
```

```python
    processor = make_processor(
        should_reorder=should_reorder,
        apply_discount=apply_discount,
        apply_tax=apply_tax,
    )
    result = processor(sample_products)

    assert result["count"] == 2
    assert pytest.approx(result["total_cost"], 0.01) == 486.0
    assert len(result["products"]) == 2

def test_compose_and_multiplier() -> None:
    discount1 = make_multiplier(0.9)
    discount2 = make_multiplier(0.95)
    combined = compose(discount2, discount1)

    assert combined(100.0) == 85.5

def test_stamp_cost_dependency_injection() -> None:
    fake_now = lambda: 1700000000.0
    cost, ts = stamp_cost(250.0, fake_now)

    assert cost == 250.0
    assert ts == 1700000000.0

def test_with_reorder_info_purity() -> None:
    product: Product = {"id": 1, "name": "Test", "category": "Cat", "stock": 1, "min_stock": 5, "uni
    new_product = with_reorder_info(product, 4, 40.0)

    assert "reorder_qty" not in product
    assert "total_cost" not in product
    assert new_product["reorder_qty"] == 4
    assert new_product["total_cost"] == 40.0
    assert new_product is not product
```

Висновок

Під час виконання лабораторної роботи №1 було засвоєно практичні навички застосування парадигми функціонального програмування у мові Python. На прикладі розробки модуля складського обліку (Варіант 12) успішно реалізовано паттерн "чисте ядро / оболонка ефектів" (Functional Core / Imperative Shell).

Відокремлення побічних ефектів (консольного виведення print, викликів системного часу) від обчислювальної логіки дозволило досягти референтної прозорості та детермінованості обчислень. Використання анотацій typing.Callable, функцій вищого порядку та композиції забезпечило гнучку параметризацію бізнес-правил (умов дозамовлення, знижок і податків) без мутації вхідних даних та без використання глобального стану. Усі створені pytest-тести та перевірка через mypy успішно пройдені, що підтверджує коректність, чистоту та типізаційну надійність розробленої системи.

```
collected 6 items

tests/test_core.py::test_referential_transparency PASSED                 [ 16%]
tests/test_core.py::test_no_mutation PASSED                              [ 33%]
tests/test_core.py::test_callable_policies PASSED                        [ 50%]
tests/test_core.py::test_compose_and_multiplier PASSED                   [ 66%]
tests/test_core.py::test_stamp_cost_dependency_injection PASSED          [ 83%]
tests/test_core.py::test_with_reorder_info_purity PASSED                 [100%]

============================================ 6 passed in 0.07s ============================================
```

```
PS C:\Users\vinit\OneDrive\Desktop\labworks\FuncProg\Lab_01> mypy core.py app.py tests/test_core.py
Success: no issues found in 3 source files
PS C:\Users\vinit\OneDrive\Desktop\labworks\FuncProg\Lab_01>
```